

Base de Datos

Cátedra José Luis Cabrera

JTP: Adrián Sergio Bernardi

Ayudantes: María Sol Fontenla, Agustina Segura.

SQL

Agenda

- Técnicas de procesamiento de consultas, usadas por los SGBD.
- Algoritmos de ordenamiento.
- Algoritmos de búsqueda.
- Algoritmos de junta.
- Algoritmos de proyección
- Algoritmos de conjuntos.
- Algoritmos para funciones de agregación.

SQL

Agenda

- Pipelining.
- Paralelización de algoritmos.
- Optimización de consultas.
- Reglas heurísticas.
- Costos.
- Plan de ejecución.

SQL

Técnicas de procesamiento de consultas

- Una consulta realizada en un lenguaje de alto nivel, se escanea, se analiza, y se valida.
- El escáner identifica las palabras clave de SQL, los nombres de atributos, los nombres de relaciones, etc.
- El analizador verifica la gramática (si está bien escrita la consulta según reglas del lenguaje).
- El validador revisa que los nombres de los atributos, relaciones, etc, tengan significado y sean válidos dentro del esquema.

SQL

Técnicas de procesamiento de consultas

- Luego se crea un árbol para la consulta.
- Se “optimiza” la consulta. En general se aproxima a una “buena” consulta, sin llegar a la consulta óptima.
- Se crea un plan de ejecución para la consulta.
- Se crea el código de la consulta, el cual es ejecutado.

SQL

Traducción al álgebra relacional

- Para optimizar una consulta, primero debemos traducirla al álgebra relacional de a bloques.
- Cada bloque posee la forma:

SELECT A

FROM B

WHERE C

GROUP BY D

HAVING E;

SQL

Algoritmos de ordenamiento

- Ordenamiento externo: ocurre al ordenar archivos en disco, que no entran completamente en memoria. Es costoso.
- Uso de índices: evita el ordenamiento en disco, si existe un índice para el/los atributos sobre los cuales se desea ordenar.

SQL

Algoritmos de búsqueda

- Permiten buscar datos en registros dentro de archivos guardados en disco, que cumplan una condición.
- Tipos de búsquedas:
 - Selección simple: escanean un archivo en búsqueda de registros que cumplan la condición dada.
 - Selección conjunta: son condiciones simples unidas con el operador AND.
 - Selección disjunta: son condiciones simples unidas con el operador OR.

SQL

Algoritmos de búsqueda

- Algoritmos para búsquedas en escenario de **selección simple**:
 - Búsqueda lineal: busca en cada registro del archivo si cumple la condición.
 - Búsqueda binaria: busca en un archivo ordenado, siempre que la comparación de la condición de búsqueda sea por igualdad.
 - Búsqueda con índice primario: se puede usar si la condición es por igualdad. Se busca ese registro puntual en el índice.
 - Búsqueda por hash: se puede usar si la condición es por igualdad. Se busca ese registro puntual en el diccionario.
 - Búsqueda con índice primario (múltiples registros): similar al tercer algoritmo, pero se pueden obtener múltiples registros.

SQL

Algoritmos de búsqueda

- Más algoritmos para búsquedas en escenario de **selección simple**:
 - Búsqueda con índice de agrupamiento (múltiples registros): similar a la búsqueda por índice primario, pero se buscan los registros en base a una condición con comparación por igualdad en campos que no son parte de una clave.
 - Búsqueda por índice secundario (árbol B+): permite buscar uno o más registros en el árbol. Las hojas del árbol son las que poseen los valores.
 - Búsqueda por índice de bits: ideal cuando se usa el operador IN, para buscar registros que incluyan valores. Se puede implementar en un vector.
 - Búsqueda por índice funcional: se crea un índice con una condición especial que aplica a una consulta dada.

SQL

Algoritmos de búsqueda

- Algoritmos para búsquedas en escenario de **selección conjunta**:
 - Usando un índice individual: se buscan las condiciones del AND individualmente en un índice y luego se verifica que se cumplan ambas para los registros obtenidos.
 - Usando un índice compuesto: se busca en un índice que posee todos los campos de la condición y se obtienen los registros.
 - Intersección de punteros: se necesitan índices secundarios con punteros a registros. Se busca la intersección de punteros en los índices (equivale a aplicar el operador AND), y luego se obtienen los registros.

SQL

Algoritmos de búsqueda

- Algoritmos para búsquedas en escenario de **selección disjunta**:
 - Sirven los 7 primeros algoritmos mostrados en selección simple, realizando la búsqueda para cada campo y uniendo los resultados.

SQL

Algoritmos de búsqueda

- Para minimizar el costo al realizar una búsqueda, el optimizador de consultas aprovecha las estadísticas del catálogo. Estas son:
 - Para cada esquema de relación (tabla):
 - Cantidad de tuplas/filas/registros.
 - El “ancho” de la relación (cuanto ocupa cada tupla físicamente).
 - Cantidad de bloques físicos que ocupa el esquema de relación.
 - El factor de bloqueo (cantidad de tuplas por bloque).
 - Para cada atributo en cada esquema de relación:
 - Valores distintos del atributo en dicho esquema de relación.
 - Valores mínimos y máximos del atributo en dicho esquema de relación.

SQL

Algoritmos de búsqueda

- Cuando el optimizador de consultas elige entre múltiples condiciones en una selección conjunta, verifica la **selectividad** de cada condición por separado.
- **Definición: Selectividad** es el factor de dividir la cantidad de tuplas que cumplen una condición, por la cantidad de tuplas del esquema de relación.
- No siempre tenemos el valor exacto de selectividad, pero se puede estimar.
- Es conveniente filtrar primero por la condición que tenga menor valor de selectividad.

SQL

Algoritmos de junta

- La junta es una de las operaciones más costosas en cuanto a tiempo de resolución de consultas.
- Las juntas más usadas son junta por igualdad, y junta natural.

SQL

Algoritmos de junta

- Hay 4 formas principales de realizar una junta entre dos tablas (archivos) (R, S):
 - Junta anidada iterativa: por cada registro en R, obtener cada registro de S y verificar si satisface la condición de junta.
 - Junta anidada iterativa con índice: obtener cada registro de R, y luego obtener los registros asociados de S usando el índice.
 - Junta por mezcla ordenada: ideal si los registros de R y S están ordenados físicamente por los atributos de la condición de junta, se escanean ambos archivos de manera concurrente obteniendo los registros.
 - Junta por partición (usando hash): Se escanea uno de los archivos (el más chico), se produce un hash (idealmente en memoria) y luego se escanea el segundo archivo, realizando la junta a medida que se recorre.

SQL

Algoritmos de junta (optimizaciones)

- Es conveniente tener un buffer de memoria amplio, para agilizar los algoritmos.
- También es conveniente que el algoritmo elija adecuadamente como hacer el join ($R \text{ join } S$ vs $S \text{ join } R$). No es lo mismo iterar R que iterar S ya que cada esquema de relación tiene (en general) distinta cantidad de tuplas.
- Utilizar índices cuando sea posible.
- Utilizar hashes cuando sea posible.

SQL

Algoritmos de proyección (y optimización)

- Si se realiza una proyección de un campo en una tabla, el algoritmo usado es buscar en cada fila y obtener ese atributo. Es decir, se realiza una búsqueda lineal, ya que no se eliminan repetidos en SQL.
- Si el/los campo/s proyectado/s incluyen una clave del esquema de relación, entonces las tuplas resultado serán todas las del esquema de relación y no hace falta usar la palabra DISTINCT.
- Si el/los campo/s proyectado/s no incluyen una clave del esquema de relación, entonces las tuplas resultado tendrán valores repetidos, y habrá que usar la palabra DISTINCT.
- En general, proyectar sólo los atributos indispensables.

SQL

Algoritmos de conjuntos (y optimizaciones)

- Las operaciones Unión, Intersección, Diferencia de conjuntos (MINUS/EXCEPT), y producto cartesiano son costosas.
- Una operación de producto cartesiano tal que R posee n tuplas y j atributos, y S posee m tuplas y k atributos, va a tener como resultado **$n \times m$ tuplas y $j + k$ atributos**.
- Evitar siempre que sea posible el producto cartesiano.
- Para Unión, Intersección, y Diferencia de conjuntos, se suele utilizar el algoritmo de “sort-merge”. Se ordenan ambos archivos (R y S) por los mismos atributos, y luego se mezclan.
- Se puede utilizar una función de hash para particionar las tablas en memoria.

SQL

Algoritmos para funciones de agregación (y optimizaciones)

- Las funciones de agregación (min, max, count, average, sum), si son aplicadas a una tabla entera, se pueden calcular recorriéndola o, si está disponible, usando un índice.

- Ejemplo:

```
SELECT min(sueldo) FROM docentes;
```

Se puede calcular tomando el primer valor del índice “sueldo_docentes” si está ordenado de manera ascendente.

- Ejemplo 2:

```
SELECT COUNT(*) FROM docentes;
```

Se puede utilizar el catálogo directamente para saber la cantidad de filas.

SQL

Algoritmos para funciones de agregación (y optimizaciones)

- Si se usa la cláusula GROUP BY, se necesita particionar la tabla en grupos, y luego calcular la función de agregación para esos grupos. En estos casos se puede ordenar cada partición y realizar el cálculo de la función de agregación en cada partición.
- Si existe un índice de agrupamiento (cluster), la partición ya la tiene el índice y se puede aprovechar.

SQL

Pipelining (canalización)

- Las consultas de SQL se pueden convertir al álgebra relacional para luego poder ser optimizadas.
- Al realizar operaciones del álgebra relacional, se pueden guardar los resultados parciales en archivos en disco. Pero esto es costoso en cuanto a espacio, y lento.
- Por eso es conveniente combinar operaciones y evitar cuando sea posible, la creación de archivos temporales grandes en disco.
- **Definición:** La combinación de operaciones en una única operación para producir un resultado se llama **pipelining**.
- Otra ventaja es que permite mostrar tuplas del resultado, a medida que se calcula el mismo.

SQL

Paralelización de algoritmos

- Existe una arquitectura llamada Paralelización de Base de Datos. Es útil en aplicaciones que utilicen exhaustivamente la base de datos.
- Hay 3 enfoques para esta arquitectura:
 - Arquitectura de memoria compartida: múltiples procesadores comparten la memoria. A mayor cantidad de procesadores, se requiere mayor contención para la memoria compartida.
 - Arquitectura de disco compartido: los procesadores comparten el disco (SAN / NAS). A mayor cantidad de procesadores, se requiere mayor contención por el ancho de banda usado.
 - Arquitectura sin recursos compartidos: cada procesador usa su memoria y disco. Es la opción más usada hoy en día.

SQL

Optimización de consultas

- El objetivo es encontrar la mejor forma de ejecutar consultas, minimizando el costo (tiempo).
- Se utilizan como herramientas:
 - Árboles de consulta.
 - Grafos de consulta.
 - Heurísticas.
- Si dos consultas son equivalentes (representan la misma búsqueda, y producen los mismos resultados), vamos a utilizar la que cueste menos obtener los datos.

SQL

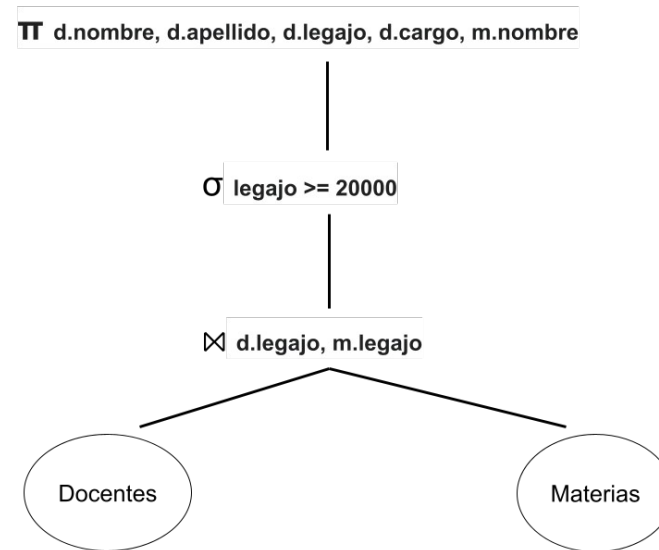
Optimización de consultas

- Un árbol de consulta representa una expresión del álgebra relacional.
- Un grafo de consulta representa una expresión del cálculo relacional.
- La intención es convertir un árbol de consulta en otro árbol de consulta equivalente, pero que éste último sea más óptimo (menor costo de consulta en cuanto a tiempo).

SQL

Optimización de consultas

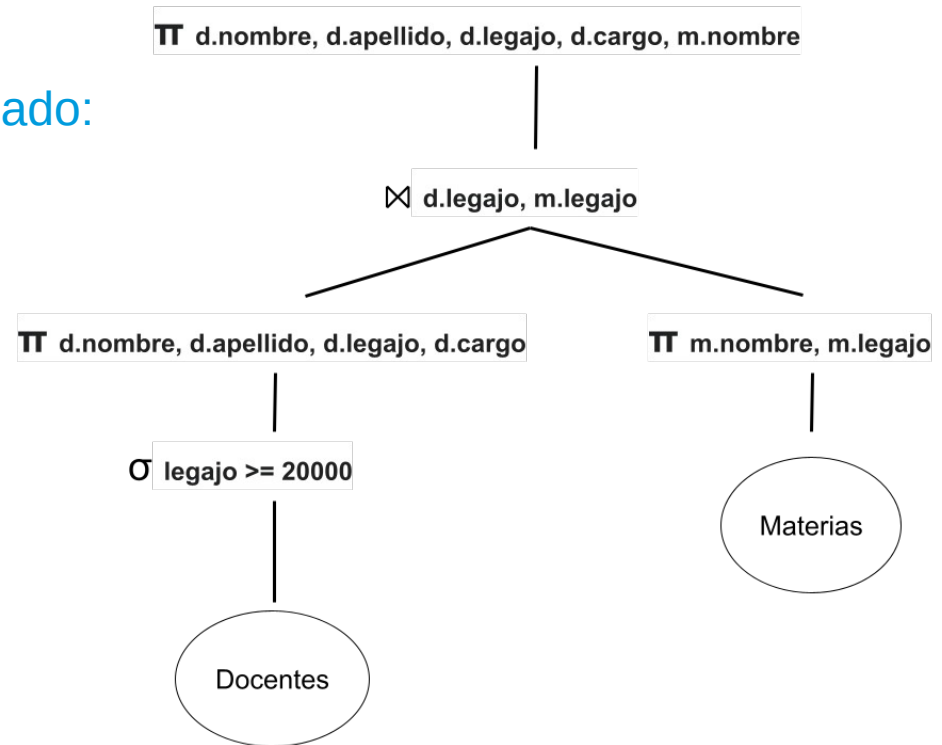
- Ejemplo de árbol de consulta:



SQL

Optimización de consultas

- Ejemplo de árbol de consulta optimizado:



SQL

Reglas heurísticas

- Cascada para selección: $\sigma_{c1 \text{ AND } c2} (R) = \sigma_{c1} (\sigma_{c2} (R))$
- Conmutatividad de σ : $\sigma_{c1} (\sigma_{c2} (R)) = \sigma_{c2} (\sigma_{c1} (R))$
- Cascada para π : $\pi_{\text{lista1}} (\pi_{\text{lista2}} (\pi_{\text{listaN}} (R))) = \pi_{\text{lista1}} (R)$
- Conmutatividad de σ con π (sólo si los atributos de la condición de σ son iguales a los atributos de la π): $\pi_{a1,a2,an} (\sigma_c (R)) = \sigma_c (\pi_{a1,a2,an} (R))$
- Conmutatividad de $\bowtie$ y X : $R \bowtie_c S = S \bowtie_c R$
- Conmutatividad de X : $R \times S = S \times R$
- Conmutatividad de σ con $\bowtie$ (sólo si la condición de selección involucra los atributos de una de las relaciones (R)): $\sigma_c (R \bowtie S) = (\sigma_c (R)) \bowtie S$

SQL

Reglas heurísticas

- Conmutatividad de π con $\bowtie$ (o $\times$): Si la proyección es la lista $a_1, a_2, \dots, a_n, b_1, b_2, \dots, b_m$ y la condición de junta involucra sólo atributos de esa lista, se pueden conmutar las operaciones proyección con junta (o proyección con producto cartesiano):
$$\pi_L (R \bowtie S) = \pi_{a_1, \dots, a_n} (R) \bowtie \pi_{b_1, \dots, b_m} (S)$$
- Conmutatividad de Unión: $R \cup S = S \cup R$
- Conmutatividad de Intersección: $R \cap S = S \cap R$
- NO ES CONMUTATIVA LA DIFERENCIA DE CONJUNTOS.
- Asociatividad de $\bowtie, \times, \cup, \cap$: $(R) \text{ op } (S \text{ op } T) = (R \text{ op } S) \text{ op } (T)$

SQL

Reglas heurísticas

- Conmutatividad de σ con union, intersección y diferencia:
 $\sigma_c (R \text{ op } S) = \sigma_c(R) \text{ op } \sigma_c(S)$
- Conmutatividad de π con unión: $\pi_L (R \cup S) = \pi_L (R) \cup \pi_L (S)$
- Convertir (producto cartesiano + selección) en una junta.
- Distribuir σ en diferencia de conjuntos: $\sigma_c (R - S) = \sigma_c (R) - \sigma_c (S)$
- Si en una condición de selección todos sus atributos pertenecen a R:
 $\sigma_c(R \cap S) = \sigma_c(R) \cap S$
- Si S está vacía: $R \cup S = R$
- Si la condición de selección es verdadera para todo R: $\sigma_c(R) = R$.

SQL

Reglas heurísticas - Síntesis

- Seleccionar lo antes posible (lo más abajo posible en el árbol de consultas).
- Seleccionar siendo lo más restrictivo posible primero, para obtener la menor cantidad de tuplas para luego usar eso en futuras operaciones.
- Proyectar lo antes posible (lo más abajo posible en el árbol de consultas).
- Evitar proyectar atributos innecesarios.
- Evitar productos cartesianos cuando sea posible.
- Cambiar productos cartesianos con selecciones, por juntas

SQL

Costos

- Costo de acceso a almacenamiento secundario (entrada/salida de datos de disco).
- Costo de almacenamiento.
- Costo computacional.
- Costo de uso de memoria.
- Costo de comunicación.

SQL

Plan de ejecución

- El plan de ejecución se basa en:
 - Árbol de consulta.
 - Índices disponibles.
 - Heurísticas.
 - Estadísticas.

SGBD Postgres:

- <https://www.postgresql.org/docs/>
- <https://www.postgresql.org/download/>

Bibliografía:

- Fundamentos de Bases de Datos (Elmasri/Navathe), Editorial Pearson.
- Fundamentos de Bases de Datos (Silberschatz/Korth/Sudarshan), Editorial McGraw-Hill.
- Sistemas de Bases de Datos, el libro completo (Garcia-Molina/Ullman/Widom), Editorial Pearson.